

PHB 243b Lecture 3: Speaker Notes

Package Management with renv and Docker

Dr. Ronald G. Thomas

2026-01-13

Table of contents

1	Course Information and Welcome	2
2	Today's Agenda Overview	2
3	The Dependency Problem	2
4	What is renv?	3
5	Core renv Workflow	4
5.1	renv::init()	4
5.2	renv::snapshot()	4
5.3	renv::restore()	4
6	The renv Lockfile	4
7	Limitations of renv	5
7.1	What renv does NOT manage:	5
8	Introduction to Docker	6
9	The Rocker Project	7
9.1	Key Rocker images:	7
10	Creating a Dockerfile	7
10.1	Instruction reference:	8
11	Running Docker Containers	8
11.1	Building	8
11.2	Running	8
11.2.1	Key flags:	9
12	Combining renv and Docker	9
12.1	Best practice workflow:	9
13	Summary and Next Steps	10
13.1	For your course projects:	10
13.2	Coming up:	10
14	Additional References	10
14.1	Primary Documentation	10

14.2 Academic Papers	10
14.3 Books	11
14.4 Package Documentation	11

1 Course Information and Welcome

Welcome to lecture 3 of PHB 243b. Today we address two essential tools for reproducible research: `renv` for R package management and Docker for containerization. These tools solve the pervasive “it works on my machine” problem that plagues collaborative data science and has contributed to the broader reproducibility crisis in computational research.

By the end of today’s lecture, you will understand why package versions matter and how to capture your computational environment for others to reproduce. The skills learned today are not merely academic exercises—they are increasingly expected in both collaborative research and industry settings. Journals such as *Nature* and *Science* now require computational reproducibility for many submissions, and funding agencies including NIH and NSF emphasize reproducibility in grant proposals.

The tools we discuss today—`renv` and Docker—represent the current best practice for R-based reproducible research. While alternatives exist (e.g., `packrat`, `conda`), the combination of `renv` and Docker provides the most robust and well-supported solution for the R ecosystem as of 2026.

Technical note: `renv` (version 1.0+) supersedes the earlier `packrat` package, which is now in maintenance mode. If you encounter older projects using `packrat`, consider migrating to `renv` using `renv::migrate()`.

2 Today’s Agenda Overview

We will proceed through six interconnected topics:

1. **The Dependency Problem in R:** Why R projects fail when moved between computers, even when the code is identical.
2. **Introduction to `renv`:** The Posit-developed solution for managing R package dependencies at the project level.
3. **Core `renv` Workflow:** The three essential functions (`init`, `snapshot`, `restore`) that form the foundation of reproducible package management.
4. **Introduction to Docker:** Containerization technology that captures the entire computing environment including the operating system and R version.
5. **Combining `renv` and Docker:** Best practices for leveraging both tools to achieve complete reproducibility.
6. **Practical Demonstration:** Hands-on application of these concepts.

The student question about code failing on a colleague’s machine illustrates the problem perfectly. The scenario described—`dplyr` version mismatch causing function name changes—is extremely common. Between `dplyr` 1.0 and 1.1, several functions were renamed or deprecated (e.g., `across()` syntax changes, `.groups` argument behavior). Such breaking changes, while improving the package long-term, create immediate reproducibility challenges.

Reference: Wickham H, et al. (2023). `dplyr`: A Grammar of Data Manipulation. R package version 1.1.4. CRAN. The NEWS file documents breaking changes between versions.

3 The Dependency Problem

The dependency problem is one of the most frustrating challenges in computational research. R’s package ecosystem is vibrant and constantly evolving—which is excellent for innovation but problematic for repro-

ducibility. As of January 2026, CRAN hosts over 21,000 packages, with thousands of updates occurring monthly.

When you write code using tidyverse functions, you are implicitly depending on a specific version of dozens of packages. The tidyverse meta-package alone loads eight core packages (ggplot2, dplyr, tidyr, readr, purrr, tibble, stringr, forcats), each with their own dependencies. A single `library(tidyverse)` call may ultimately load 50+ packages.

If any of these packages changes its interface, your code may break. This is not hypothetical—it happens regularly. A 2018 study by Trisovic et al. found that only 25% of R code from published papers could be successfully executed without modification, with package version issues being a leading cause of failure.

The student question about why package authors make breaking changes addresses a fundamental tension in software development. Package authors face a tradeoff:

- **Maintaining backward compatibility forever** leads to accumulated technical debt, confusing APIs, and inability to fix design flaws.
- **Making breaking changes** improves the package but disrupts existing users.

Most R package authors choose improvement, following semantic versioning conventions where major version changes (e.g., 1.x to 2.x) signal potential breaking changes. The tidyverse team has been particularly thoughtful about this, introducing lifecycle warnings (deprecated, superseded, experimental) to telegraph upcoming changes.

Technical note: The `lifecycle` package provides infrastructure for managing function deprecation. When you see warnings like “This function was deprecated in dplyr 1.0.0,” this system is at work. These warnings typically persist for two major versions before functions are removed entirely.

Reference: Trisovic A, et al. (2022). A large-scale study on research code quality and execution. *Scientific Data* 9:60.

4 What is renv?

The `renv` package, developed by Kevin Ushey at Posit (formerly RStudio), solves the package versioning problem by giving each R project its own private library of packages. The name stands for “Reproducible ENVironments,” and it delivers on that promise through three mechanisms:

1. **Project isolation:** Instead of all your projects sharing the same installed packages in a user or system library, each project maintains its own collection in a `renv/library` subdirectory.
2. **Version recording:** When you “snapshot” your project, `renv` creates a lockfile (`renv.lock`) that records the exact version of every package, including the repository source and a content hash.
3. **Environment restoration:** The lockfile can be shared with collaborators or used to recreate your environment months or years later via `renv::restore()`.

The student question about how `renv` differs from manual documentation highlights several key advantages:

- **Automation:** `renv` automates installation rather than relying on manual `install.packages()` calls.
- **Dependency resolution:** `renv` handles transitive dependencies (package A requires package B version ≥ 1.2), which humans routinely forget.
- **Source tracking:** `renv` records whether packages came from CRAN, GitHub, Bioconductor, or local sources, ensuring identical reinstallation.
- **Hash verification:** The content hash detects if a package has been modified since the snapshot.

Technical note: `renv` uses a global package cache (`~/.local/share/renv` on Linux/macOS, `%LOCALAPPDATA%/renv` on Windows) to avoid downloading the same package version multiple times.

When you restore a project, `renv` first checks the cache and only downloads packages not already present. This dramatically speeds up restoration for projects with overlapping dependencies.

Reference: Ushey K. (2023). `renv`: Project Environments for R. R package version 1.0.3. <https://rstudio.github.io/renv/>

5 Core `renv` Workflow

The `renv` workflow centers on three functions that form a complete lifecycle for package management:

5.1 `renv::init()`

Initializes `renv` for your project, creating the necessary infrastructure:

- `.Rprofile` file that activates `renv` when the project opens
- `renv/` directory containing the project-specific library
- Initial `renv.lock` file capturing current package state

You typically run this once at project creation. For existing projects, `init()` will discover packages used in your R scripts and install them into the project library.

5.2 `renv::snapshot()`

Captures your current package state into the lockfile. Run this whenever you:

- Install new packages
- Update existing packages
- Remove packages you no longer need

Best practice is to snapshot before committing to Git, ensuring the lockfile stays synchronized with your code. Some teams snapshot daily as a habit.

5.3 `renv::restore()`

Reads the lockfile and installs the exact package versions recorded there. Your collaborators run this after cloning your repository to get identical packages. This is also useful for reverting your own environment after experimental changes.

The lockfile is the critical artifact—it should be committed to version control and shared with your team. The `renv/library/` directory should NOT be committed (add it to `.gitignore`), as it contains platform-specific binaries.

Technical note: `renv` provides additional useful functions:

- `renv::status()`: Compare installed packages to lockfile, identify discrepancies
- `renv::update()`: Update packages to latest versions (then snapshot)
- `renv::clean()`: Remove packages not recorded in the lockfile
- `renv::history()`: View previous lockfile versions (if using Git)

Reference: See the `renv` documentation at <https://rstudio.github.io/renv/> for comprehensive coverage of all functions and options.

6 The `renv` Lockfile

The `renv.lock` file is a JSON document that completely describes your package environment. Understanding its structure helps with troubleshooting:

```
{
  "R": {
    "Version": "4.4.0",
    "Repositories": [
      {"Name": "CRAN", "URL": "https://cloud.r-project.org"}
    ]
  },
  "Packages": {
    "dplyr": {
      "Package": "dplyr",
      "Version": "1.1.4",
      "Source": "Repository",
      "Repository": "CRAN",
      "Hash": "597b13f9a7f5c2e3f8ab..."
    }
  }
}
```

Key fields:

- **R.Version:** The R version used when snapshotting. `renv` warns (but does not prevent) restoration if versions mismatch.
- **Repositories:** Where packages were downloaded from. Supports CRAN, Bioconductor, and custom repositories.
- **Package entries:** For each package, records version, source, repository, and a SHA-256 hash of the package contents.

The student question about why hashes are included addresses an important security and integrity concern. The hash serves multiple purposes:

1. **Integrity verification:** Ensures the installed package matches exactly what was recorded, detecting corruption during download or storage.
2. **Tampering detection:** If someone modified a package on CRAN (which should not happen but theoretically could), the hash mismatch would alert you.
3. **Cache validation:** Allows `renv` to verify that cached packages are correct before symlinking them into the project library.

Technical note: You should never edit `renv.lock` manually—always use `renv` functions. Manual edits bypass integrity checks and may cause subtle failures. If you need to force a specific package version, use `renv::install("package@version")` followed by `renv::snapshot()`.

7 Limitations of `renv`

While `renv` is excellent for package management, it has important limitations that motivate the use of Docker:

7.1 What `renv` does NOT manage:

- **R version:** The lockfile records the R version but cannot install a different R version for you. If your analysis requires R 4.3 but your collaborator has R 4.4, `renv` will warn them but cannot fix it.
- **Operating system:** Some R code behaves differently on Windows versus macOS versus Linux due to path separators, encoding, or system calls.

- **System libraries:** Many R packages depend on system libraries (libcurl, libxml2, libpng, GDAL, etc.). These must be installed separately on each machine, often with version-specific requirements.
- **Other software:** Pandoc (for R Markdown), LaTeX (for PDF output), Python (for reticulate), databases, etc. are outside renv’s scope.

The student question about system library differences provides a perfect example. The `sf` package for spatial analysis requires three major system libraries:

- **GDAL** (Geospatial Data Abstraction Library): For reading/writing spatial data formats
- **GEOS** (Geometry Engine Open Source): For geometric operations
- **PROJ:** For coordinate transformations

Different versions of these libraries can produce slightly different results for coordinate transformations or geometric calculations. This is why spatial analyses sometimes produce different results on different machines even with identical R packages.

Technical note: To check which system libraries an R package requires, examine the `SystemRequirements` field in its `DESCRIPTION` file or use `tools::package_dependencies("package", which = "all")`. For `sf` specifically, running `sf::sf_extSoftVersion()` displays the versions of linked system libraries.

Reference: The renv documentation section on “System Requirements” discusses these limitations: <https://rstudio.github.io/renv/articles/faq.html>

8 Introduction to Docker

Docker revolutionized software deployment by introducing containers— self-contained environments that package everything needed to run an application. For data science, a container includes:

- The operating system (typically a minimal Linux distribution)
- The R installation (specific version)
- All R packages (installed via renv)
- System libraries (GDAL, curl, etc.)
- Your analysis code and data

When you run a Docker container, it behaves identically regardless of the host machine. A container built on your Mac will run the same on a Windows PC, a Linux server, or a cloud computing platform (AWS, Google Cloud, Azure). This eliminates environment-related bugs and makes sharing analyses trivial.

The student question about how containers differ from virtual machines highlights a key architectural distinction:

- **Virtual machines** include a complete operating system with its own kernel, managed by a hypervisor. They are heavy (gigabytes), slow to start (minutes), and resource-intensive.
- **Containers** share the host machine’s kernel and only package the differences from a base image. They are lightweight (megabytes to low gigabytes), start in seconds, and use minimal overhead.

For data science, this efficiency means you can quickly spin up reproducible environments without the overhead of full virtualization. You might have dozens of project-specific containers on a single machine.

Technical note: Docker uses a layered filesystem where each Dockerfile instruction creates a new layer. Unchanged layers are cached and reused, making rebuilds fast. Understanding this caching behavior is key to writing efficient Dockerfiles.

Reference: Merkel D. (2014). Docker: lightweight Linux containers for consistent development and deployment. *Linux Journal* 2014(239):2.

9 The Rocker Project

The Rocker project, maintained by Carl Boettiger and Dirk Eddelbuettel, provides pre-built Docker images specifically for R users. These images save tremendous time because you do not need to configure R installation from scratch.

9.1 Key Rocker images:

Image	Contents	Use Case
<code>rocker/r-ver:4.4.0</code>	R version 4.4.0 only	Minimal base for custom images
<code>rocker/rstudio</code>	R + RStudio Server	Interactive development in browser
<code>rocker/tidyverse</code>	R + RStudio + tidyverse	Quick starts with common packages
<code>rocker/verse</code>	tidyverse + LaTeX + publishing	R Markdown/Quarto documents
<code>rocker/geospatial</code>	verse + sf, terra, GDAL	Spatial analysis
<code>rocker/ml</code>	verse + ML packages	Machine learning workflows

The student question about choosing `r-ver` over `tidyverse` addresses an important reproducibility principle: **minimize your dependencies**. Starting with the minimal `r-ver` image and adding only needed packages creates:

- **Smaller images:** Less disk space and faster downloads
- **Explicit dependencies:** Every package is documented in your Dockerfile
- **Reduced attack surface:** Fewer packages means fewer potential vulnerabilities
- **Stability:** Pre-built images like `tidyverse` may change over time

For reproducibility, it is better to explicitly install and document every dependency rather than rely on a large pre-built image whose contents may evolve between versions.

Technical note: Rocker images use version tags corresponding to R versions (e.g., `rocker/r-ver:4.4.0`). The `latest` tag points to the most recent R release. For reproducibility, always specify an explicit version tag rather than using `latest`.

Reference: Boettiger C, Eddelbuettel D. (2017). An Introduction to Rocker: Docker Containers for R. *The R Journal* 9(2):527-536.

10 Creating a Dockerfile

A Dockerfile is a recipe for building a Docker image. Each line is an instruction that adds a layer to the image. Here is a breakdown of a typical Dockerfile for R analysis:

```
# Start from R 4.4.0
FROM rocker/r-ver:4.4.0

# Set working directory
WORKDIR /workspace

# Install renv
RUN R -e "install.packages('renv', repos='https://cloud.r-project.org')"

# Copy lockfile and restore packages
COPY renv.lock /workspace/renv.lock
RUN R -e "renv::restore()"

# Copy analysis files
```

```
COPY . /workspace
```

```
# Default command
```

```
CMD ["R"]
```

10.1 Instruction reference:

- **FROM:** Specifies the base image. Always start from an established image rather than building from scratch.
- **WORKDIR:** Sets the working directory for subsequent commands. Creates the directory if it does not exist.
- **RUN:** Executes commands during image building. Each RUN creates a new layer.
- **COPY:** Transfers files from your computer (the “build context”) into the image.
- **CMD:** Specifies the default command when the container starts. Can be overridden at runtime.

The student question about copying the lockfile before other files addresses Docker’s layer caching mechanism. Docker caches each layer and only rebuilds when something changes. If we copy all files first:

1. Any change to analysis code invalidates the COPY layer
2. This invalidates all subsequent layers
3. Package installation (slow) must be repeated

By copying the lockfile separately:

1. Changes to analysis code only invalidate the final COPY
2. Package layers remain cached
3. Rebuilds are fast during development

Technical note: You can further optimize by copying `renv.lock` and the `renv/activate.R` file together, then running `restore`. For system dependencies, install them in a separate RUN instruction before R packages so they remain cached when packages change.

11 Running Docker Containers

Building and running Docker containers involves several commands and flags:

11.1 Building

```
docker build -t my-analysis:v1 .
```

- **build:** Creates an image from a Dockerfile
- **-t my-analysis:v1:** Tags the image with a name and version
- **..:** Build context (current directory)

11.2 Running

```
docker run --rm -it \
-v "$(pwd):/workspace" \
-w /workspace \
my-analysis:v1 \
/bin/bash
```


11.2.1 Key flags:

- **--rm**: Automatically remove the container when it exits (cleanup)
- **-it**: Interactive terminal (allocate TTY and keep STDIN open)
- **-v "\$(pwd):/workspace"**: Mount current directory into container
- **-w /workspace**: Set working directory inside container

The student question about file persistence is critical for understanding container behavior. Without a volume mount:

1. The container has its own isolated filesystem
2. Files created inside exist only in that filesystem
3. When the container stops (especially with **--rm**), files vanish

This ephemeral nature is actually a feature:

- **Reproducibility**: Containers start fresh every time, ensuring identical initial conditions
- **Isolation**: Container activities cannot affect the host system
- **Security**: Malicious code cannot persist on the host

During development, however, you need mounts to:

- Edit code on your host and see changes in the container
- Save analysis outputs that persist after the container stops
- Access data files stored on your host

Technical note: Volume mounts create a direct link between host and container filesystems. Changes on either side are immediately visible to the other. For read-only data, use **-v /path/to/data:/data:ro** to prevent accidental modification.

12 Combining renv and Docker

The combination of renv and Docker provides comprehensive reproducibility that neither tool achieves alone:

- **Docker guarantees**: OS version, R version, system libraries, software versions
- **renv guarantees**: Exact R package versions, sources, and integrity

12.1 Best practice workflow:

1. **Develop locally with renv**: Work normally in your preferred environment, using renv to manage packages.
2. **Snapshot regularly**: Run `renv::snapshot()` after package changes, commit the lockfile to Git.
3. **Create Dockerfile**: When ready to share, create a Dockerfile that builds on a Rocker image and restores packages from your lockfile.
4. **Build and test**: Build the image locally, run your analysis inside it, verify results match your local environment.
5. **Share via registry**: Push the image to DockerHub, GitHub Container Registry, or an institutional registry.
6. **Collaborators pull and run**: Others pull your image and run the analysis without installing anything except Docker.

The student question about commit order for new projects suggests this sequence:

1. **Initial commit**: Basic project structure, `.gitignore`
2. **After `renv::init()`**: Add `renv.lock`, `.Rprofile`, `renv/activate.R`
3. **During development**: Update `renv.lock` as packages change

4. **When stable:** Add `Dockerfile` and build instructions
5. **Documentation:** README explaining both `renv` and Docker workflows

Technical note: For projects that might be used without Docker, maintain both pathways. Document how to use `renv::restore()` for local installation and how to use Docker for complete reproducibility. Include the `renv` cache directory in `.dockerignore` to avoid copying it into images.

Reference: The “Reproducible Research with R and Docker” chapter in *R for Data Science* (2nd ed.) by Wickham and Grolemund provides additional guidance.

13 Summary and Next Steps

Today we covered two essential tools for reproducible research:

1. **The Problem:** R projects break due to package version mismatches, system library differences, and R version incompatibilities. This is a major contributor to the reproducibility crisis in computational research.
2. **renv:** Solves package management by creating project-specific libraries and recording exact versions in a lockfile. The three-function workflow (`init`, `snapshot`, `restore`) is straightforward to adopt.
3. **Docker:** Solves environment portability by containerizing everything from the operating system up. The Rocker project provides R-specific base images that simplify adoption.
4. **Combined workflow:** Develop with `renv` locally, deploy with Docker for sharing. This combination eliminates the “works on my machine” problem.

13.1 For your course projects:

- **Required:** Use `renv` to manage dependencies. Commit `renv.lock` to your repositories.
- **Recommended:** Create Dockerfiles for final project submissions, especially for analyses intended to be shared beyond the class.

13.2 Coming up:

- **Lecture 4 (Thursday):** Git and GitHub for version control and collaboration. We will cover branching, merging, pull requests, and collaborative workflows.

Technical note: If you have not already, install Docker Desktop on your machine before Thursday. We will use it in upcoming demonstrations. Docker Desktop is free for educational use: <https://www.docker.com/products/docker-desktop/>

14 Additional References

14.1 Primary Documentation

- `renv` documentation: <https://rstudio.github.io/renv/>
- Docker documentation: <https://docs.docker.com/>
- Rocker Project: <https://rocker-project.org/>

14.2 Academic Papers

- Boettiger C, Eddelbuettel D. (2017). An Introduction to Rocker: Docker Containers for R. *The R Journal* 9(2):527-536. <https://doi.org/10.32614/RJ-2017-065>
- Trisovic A, Lau MK, Pasquier T, Crosas M. (2022). A large-scale study on research code quality and execution. *Scientific Data* 9:60. <https://doi.org/10.1038/s41597-022-01143-6>

- Stodden V, Seiler J, Ma Z. (2018). An empirical analysis of journal policy effectiveness for computational reproducibility. *PNAS* 115(11):2584-2589. <https://doi.org/10.1073/pnas.1708290115>

14.3 Books

- Bryan J, Hester J. *Happy Git and GitHub for the useR*. <https://happygitwithr.com/>
- Wickham H, Bryan J. *R Packages* (2nd ed.). <https://r-pkgs.org/>
- Wickham H, Golemund G. *R for Data Science* (2nd ed.). <https://r4ds.hadley.nz/>

14.4 Package Documentation

- renv NEWS (changelog): <https://rstudio.github.io/renv/news/index.html>
- Rocker versioning: <https://rocker-project.org/images/>
- lifecycle package: <https://lifecycle.r-lib.org/>